Alsmering

AI-Driven Online Proctoring & Anti-Cheat System

Software Design Document

Version 01.00

Revision History

Date	Version	Description	Author
09/05/2026	01.00	Initial draft	Cabahug, Bacusmo, Vargas, Caneja

Table of Contents

1. INTRODUCTION 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Overview 4
   - 1.4 Reference Material 5
   - 1.5 Definitions and Acronyms 5
2. SYSTEM OVERVIEW 6
3. SYSTEM ARCHITECTURE 7
   - 3.1 Architectural Design 7
   - 3.2 Decomposition Description 8
   - 3.3 Design Rationale 9
4. DATA DESIGN 10
   - 4.1 Data Description 10
   - 4.2 Data Dictionary 12
   - 4.3 Entity Relationship Diagram 13
5. COMPONENT DESIGN 14
   - 5.1 Test-Taker Client - Capture, Calibration, and Exam Interface 14
   - 5.2 CV/ML Inference Services - Detection and Classification 14
   - 5.3 Stream Processor - Fusion and Risk Scoring 15
   - 5.4 Storage Layer - Persistence and Retention 15
   - 5.5 LLM Summarization Service - Advisory Session Narratives 15
   - 5.6 Proctor Dashboard - Review Workflow 16
   - 5.7 Exam Lifecycle Management and Retention Job 17
   - 5.8 Proctor Database Viewer and Data Management - Mock/Test Data Administration 17
   - 5.9 Proctor Dashboard - Optional Live View (Future Enhancement) 18
6. HUMAN INTERFACE DESIGN 18
   - 6.1 Overview of User Interface 18
   - 6.2 Screen Images 20
   - 6.3a Screen Objects and Actions 23
   - 6.3b Screen Objects and Actions (Addendum) 24
7. REQUIREMENTS MATRIX 25

Software Design Document

1. INTRODUCTION

1.1 Purpose

This software design document describes the architecture and design of the AI-Driven Online Proctoring & Anti-Cheat System. It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS v1.2) into a concrete technical design covering the system architecture, data design, component design, human interface design, and requirements mapping. The intended audience is the development team, quality assurance engineers, and project stakeholders who will build, verify, and maintain the system.

1.2 Scope

This document defines the complete technical design for the AI-Driven Online Proctoring & Anti-Cheat System. It covers the system architecture (Section 3), data design and data dictionary (Section 4), component design with procedural descriptions of the central algorithms (Section 5), human interface design (Section 6), and the requirements traceability matrix mapping each SRS requirement to its satisfying component (Section 7). It translates the functional and non-functional requirements in the SRS (v1.2) into concrete design decisions for the test-taker monitoring client, the computer vision and audio machine learning services, the event-driven data pipeline and storage layer, the LLM-based session summarizer, user and account management, examination management, the online examination experience, and examination and proctoring reports, along with the privacy, security, and retention mechanisms required for FERPA/GDPR compliance.

Consistent with the SRS, this document does not specify the authoring of exam question content, the grading or scoring of exam responses, or identity verification at account-creation time, since these fall outside the boundaries of the system being designed. It also does not cover project scheduling, staffing, or budget, which are documented separately in the Software Project Management Plan. Version 01.01 adds the design of three proctor-side enhancements: export of completed student test recordings, an optional live view of a student actively taking a test (documented as a future enhancement), and a proctor database viewer with guarded delete operations for mock/test data. These are specified in Sections 3.2, 4.1, 5.6, 5.8, 5.9, 6, and 7.

1.3 Overview

Section 2 gives a general description of the system and its design context. Section 3 presents the system architecture: the architectural design (3.1), the decomposition description (3.2), and the design rationale (3.3). Section 4 describes the data design, including the data dictionary. Section 5 details the component design with procedural descriptions of the central algorithms. Section 6 describes the human interface design, and Section 7 provides the requirements matrix that traces components to the requirements in the SRS. Sections 3.2, 4.1, 5, 6, and 7 also describe the proctor-side enhancements added in version 01.01: export of completed-session recordings, the optional live student view (future enhancement), and the proctor database viewer with guarded delete operations for mock/test data.

1.4 Reference Material

- Software Requirements Specification, AI-Driven Online Proctoring & Anti-Cheat System, Version 1.2
- FERPA (20 U.S.C. 1232g)
- GDPR (Regulation (EU) 2016/679)
- Internal design notes / prior discussion on system architecture

1.5 Definitions and Acronyms

Term	Definition
CV	Computer Vision
ML	Machine Learning
LLM	Large Language Model
FERPA	Family Educational Rights and Privacy Act
GDPR	General Data Protection Regulation
Gaze Vector	Estimated direction of a person's visual attention
Pitch/Yaw/Roll	The three rotational axes used to describe head orientation
Risk Score	A composite numeric indicator of a test-taker's likelihood of cheating
Multimodal Fusion	Combining signals from multiple input types (e.g., audio + video) to make a single decision
Session Summary	A natural-language, LLM-generated recap of a test-taker's flagged events during a session
Message Queue	A durable buffer (e.g., Kafka/Kinesis) that decouples event producers from consumers and absorbs load spikes
Object Storage	A storage backend that holds encrypted audio/video recordings; the database stores only a reference pointer to each object
PDL	Program Design Language - the pseudocode notation used in Section 5
Stream Processor	The service that consumes queued events, performs multimodal fusion, and maintains composite risk scores
Presigned URL	A short-lived, authorization-bearing download link generated by object storage that grants temporary access to one recording object.
Mock/Test Data	Seed or demonstration records created for testing and evaluation; the only records the proctor may delete through the database viewer.

Live View (optional)	Optional proctor-side enhancement that displays a student's camera feed or periodic snapshots while the student is taking a test (future enhancement).

2. System Overview

The system is an AI-driven remote exam monitoring platform that detects potential cheating in real time while keeping a human proctor in every final decision. During an exam, a browser- or client-based test-taker application continuously captures webcam video, microphone audio, and browser-level behavioral signals. These signals are streamed to backend machine-learning services that estimate head pose, track gaze, classify audio, and detect unauthorized objects or additional people. Detected anomalies are published as structured events and fused into a composite risk score that proctors monitor on a real-time dashboard.

Flagged events never trigger automatic penalties. A proctor reviews each flag - acknowledging and dismissing it, escalating it to an academic integrity case, adding a reviewer note, overriding its severity, or bulk-reviewing low-severity flags - and the audit trail preserves both the original evidence and every human action. After a session, or on demand, an LLM-based summarization service converts the structured event log into a plainlanguage, timestamped narrative that proctors can cross-check against the raw flags; the summary is advisory only and never a finding in itself. Recordings of completed sessions can be played back in the dashboard and exported by the proctor, with each export recorded in the audit trail.

The platform also manages the surrounding examination lifecycle: role-based accounts and institution tenants; exam creation, scheduling, roster assignment, and proctoring policy templates; the online exam experience itself (check-in, calibration, exam interface, integrity notices, and technical-support escalation); and post-exam reporting and analytics. Version 01.01 additionally provides a proctor database viewer for inspecting records and deleting mock/test data under guardrails, and documents an optional live view of students taking a test as a future enhancement (Sections 3.2, 5.8, 5.9).

The design organizes this functionality into ten cooperating subsystems, described in Section 3.2, backed by a relational data model and encrypted object storage described in Section 4. Because the system operates under strict privacy and biometric-data regulations (FERPA/GDPR), encryption, data minimization, retention enforcement, and auditability are treated as first-class architectural concerns rather than add-ons.

3. System Architecture

3.1 Architectural Design

The architecture is a layered, event-driven design in which a thin capture client, stateless machine-learning services, an asynchronous event pipeline, and a relational/object storage tier are decoupled by message queues and APIs. Figure 3-1 shows the major subsystems, data repositories, and their interconnections.

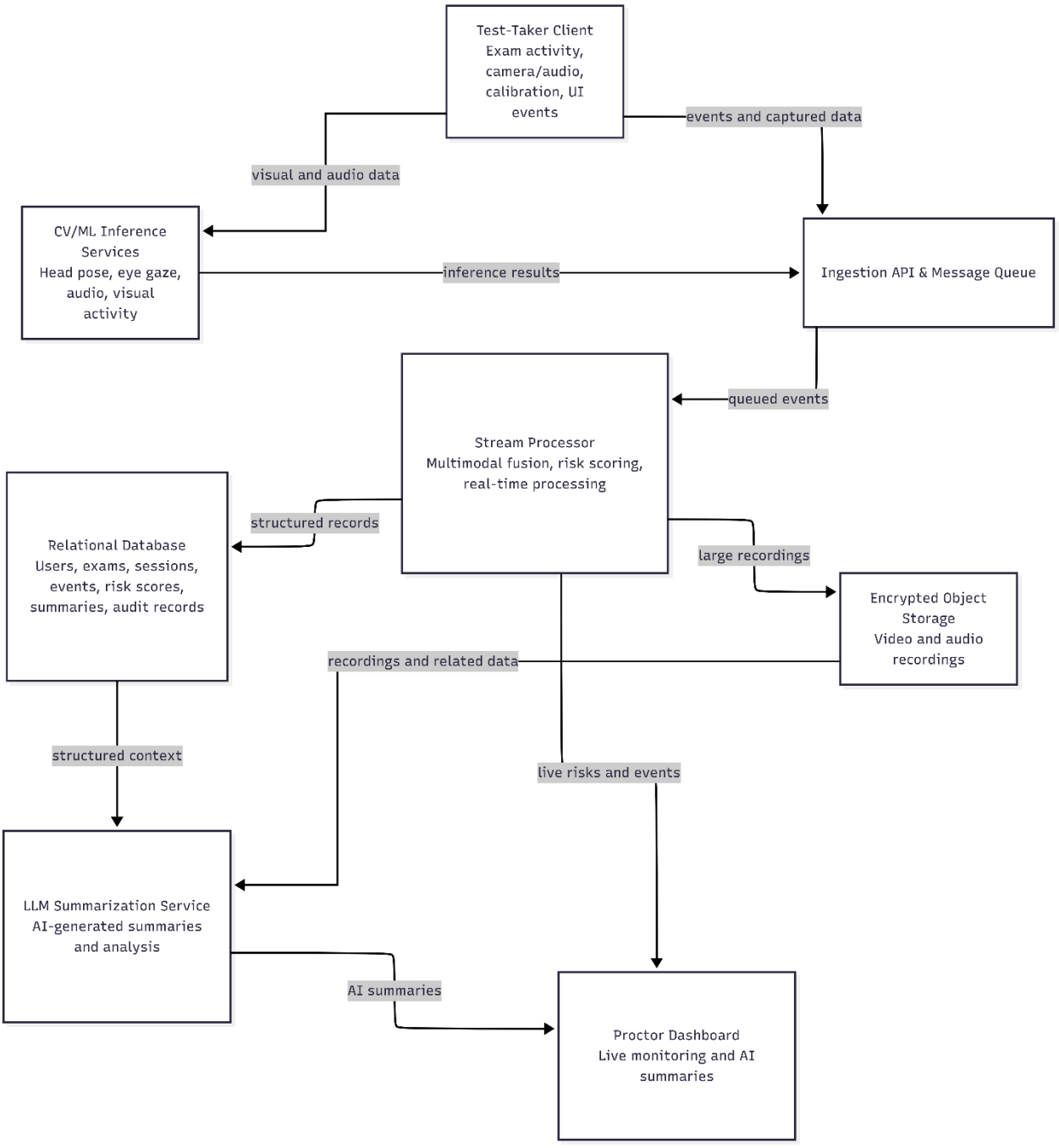


Figure 3-1. System architecture: major subsystems, data repositories, and interconnections. Solid arrows show the primary data flows; the test-taker client also streams encrypted recordings directly to object storage (see Section 4.1).

- Client Layer - Test-Taker Client (webcam/microphone/behavior capture, calibration, exam interface) and Proctor Dashboard (live risk scores, flag review, AI summaries, recording playback and export, database viewer).
- API and Ingestion Layer - REST/WebSocket ingestion API and message queue that absorb event bursts from concurrent exam sessions.
- Processing Layer - CV/ML inference services (head pose, gaze, acoustic and visual classification), the Stream Processor (multimodal fusion, risk scoring, persistence), and the LLM Summarization Service.
- Data Layer - relational database (Users, Exams, ExamSessions, Events/Flags, RiskScores, Recordings, SessionSummaries, audit and retention tables) plus encrypted object storage for audio/video recordings.
- Management Layer - Exam Lifecycle Management, User and Account Management, the Reporting Service, and the scheduled Retention/Purge job.

Primary data flow: the client publishes captured signals, browser behavior events, and ML flags to the ingestion API; the message queue feeds the stream processor, which fuses multimodal signals (SRS 3.1.2), recomputes the composite risk score, and persists structured events and scores to the database while recordings stream to encrypted object storage. The proctor dashboard receives live score and flag updates. On exam completion or on demand, the LLM summarization service reads only the structured Events/Flags rows for a session, generates a chronological timestamped narrative, and stores it in SessionSummaries. Recording media never enters the relational database, and the LLM never receives raw media or biometric feature data. Completed recordings are exported through short-lived presigned URLs generated from the Recordings rows, so media still never passes through the relational database. An optional live view of an active session (Section 5.9) is a deferred enhancement and is not part of the core data flow.

3.2 Decomposition Description

The system is decomposed into ten subsystems. Each subsystem is described below with its responsibilities and its principal inputs and outputs.

- Test-Taker Client - captures webcam video, microphone audio, and browser-level behavioral events during the exam; handles consent capture, device check, camera/microphone calibration, and baseline capture; presents the exam interface, timer, and non-punitive integrity notices. Inputs: media streams and browser events. Outputs: structured signals and ML flags to the ingestion API; encrypted recordings to object storage.
- CV/ML Inference Services - stateless services that estimate head pose and gaze (SRS 3.1.1) and classify audio and visual inputs (SRS 3.1.2). Inputs: video frames and audio chunks. Outputs: structured detections with confidence values, passed to the client for publication.
- Event Ingestion API and Message Queue - accepts events over REST/WebSocket and enqueues them durably, absorbing load spikes across concurrent exam sessions (SRS 3.1.4). Inputs: signal, behavior, and ML flag events. Outputs: ordered queue of events.

- Stream Processor - consumes queued events, performs multimodal fusion, computes and updates the composite risk score, and persists results (SRS 3.1.4). Inputs: queue events. Outputs: Events/Flags and RiskScores rows; live updates to dashboards.
- Storage Layer - relational schema described in Section 4 plus encrypted object storage for recordings; enforces retention via a scheduled purge job (SRS 3.1.4, 3.6). Inputs: structured rows and media objects. Outputs: query, playback, and export access for dashboards, reports, and the summarizer.
- LLM Summarization Service - reads structured event rows for a session and generates a chronological, timestamped, advisory-only narrative (SRS 3.1.5). Inputs: Events/Flags rows (nonbiometric fields only). Outputs: SessionSummaries rows.
- Proctor Dashboard - real-time multi-session monitoring with prioritized risk scores and flags; supports acknowledge and dismiss, escalate to academic integrity case, reviewer notes, severity override, and bulk review of low-severity flags (SRS 3.1.3); shows the AI summary panel clearly labeled and distinct from raw flags; plays back and exports recordings of completed sessions; includes a read-only database viewer with guarded delete operations for mock/test data; and optionally offers a live view of active sessions (future enhancement, Section 5.9). Inputs: live score/flag updates and query results. Outputs: review actions, recording export requests, and database viewer actions recorded in the audit trail.
- User and Account Management - role-based accounts (Student, Instructor/Proctor, Administrator, Appeals Reviewer), institution tenant provisioning, roster import, SSO (SAML/OAuth), and consent management (SRS 3.1.6). Inputs: provisioning and authentication requests. Outputs: user accounts, roles, consents.
- Examination Management and Online Exam Delivery - exam creation and configuration, proctoring policy templates, roster assignment and accommodations, the exam lifecycle state machine, and the student exam experience including check-in and technical-support escalation (SRS 3.1.7, 3.1.8). Inputs: instructor configuration and student activity. Outputs: exam records, session records, and state transitions.
- Reporting and Compliance Service - session integrity reports, class/cohort analytics, appeals package exports, and audit/compliance reports (SRS 3.1.10). Inputs: stored events, scores, summaries, and audit rows. Outputs: PDF/CSV reports and export bundles.

Proctor-side enhancements (version 01.01). Three proctor-side capabilities are added to the Proctor Dashboard subsystem and designed in Section 5: recording export for completed sessions (Section 5.6), the database viewer with guarded delete operations (Section 5.8), and the optional live view of active sessions (Section 5.9, future enhancement). No new subsystem is introduced.

3.3 Design Rationale

Event-driven pipeline. Captured signals and flags are published asynchronously to a message queue instead of being processed synchronously. This decouples capture from analysis, absorbs load spikes when many sessions run at once (SRS 3.4), and makes every event an immutable record that supports audit and replay. The alternative - synchronous request/response flagging - was rejected because client latency would grow with ML load and a burst of sessions could overwhelm the processing tier.

Human-in-the-loop enforcement. Every flag and every AI summary is advisory; the architecture routes all findings through the proctor review workflow, records every human action in the audit trail, and retains original

severity values when overridden (SRS 3.6). Fully automated accusation was rejected: false positives on highstakes exams are unacceptable, and FERPA/GDPR scrutiny requires accountable human decisions.

Separation of media from metadata. Raw recordings are stored as encrypted objects in object storage, and the relational database keeps only reference pointers (SRS 3.1.4). This keeps the relational store small and fast, simplifies encryption and retention, and avoids the backup and size costs of storing media in the database.

LLM data minimization. The summarization service sends only structured, non-biometric fields (student_id, timestamp, signal_type/error_type, confidence, duration) to the LLM; raw media and biometric feature data are excluded by design (SRS 3.5, 3.6). Sending raw media or transcripts was rejected on privacy and cost grounds.

Stateless, horizontally scalable processing. Inference services and the stream processor are stateless, so they scale out with the number of concurrent sessions. If a real-time ML service becomes unavailable, the system degrades gracefully to recording-only mode rather than failing the exam (SRS 3.3). A monolithic application was rejected because it would create a single point of failure and scale poorly across exam windows.

Proctor-side recording export. A proctor downloads a completed recording through a short-lived presigned URL generated from the existing Recordings pointer row, so the media itself never enters the relational database (Section 4.1). Every export is an audit-logged proctor action, consistent with the human-in-the-loop enforcement model (SRS 3.5).

Live monitoring deferred as an optional enhancement. Continuously streaming every active session to proctor dashboards would multiply bandwidth, latency, and server load (SRS 3.4), so the live view is specified as a future enhancement (Section 5.9) rather than a core capability. Candidate mechanisms - WebRTC peer-to-peer streaming, WebSocket-based frame push, and periodic low-frequency snapshots - are documented as options, and the final choice is deferred until per-session bandwidth and server load are measured.

Guarded data management. Proctor access to raw records is read-only by default; deletion is limited to mock/test data with explicit confirmation and audit logging, and the scheduled retention job remains the only production deletion path (SRS 3.6).

4. DATA DESIGN

4.1 Data Description

The information domain of the system has three kinds of data: reference/configuration data (users, institutions, exams, policies, rosters), time-series monitoring data (signals, flagged events, risk scores), and heavy media (encrypted audio/video recordings). Reference data lives in normalized relational tables; monitoring data is stored as append-only structured rows; media never enters the relational database - only encrypted objects with reference pointers.

The relational schema is organized into four groups. Identity and exam lifecycle tables: UserAccount, Institution, Exam, ExamSession, ProctoringPolicy, RosterAssignment, and Consents. Monitoring tables: EventsFlags (one row per detected behavioral, ML, or reliability event: session_id, student_id, timestamp, signal_type/error_type, confidence, duration, severity, review status) and RiskScores (session_id, running score, last_updated). Media and AI tables: Recordings (session_id, object_key, format, encryption metadata) and SessionSummaries (session_id, summary_text, generated_at, model_version). Governance tables: AuditLog (who viewed or acted on what) and RetentionLog (purge actions).

Key relationships follow the SRS core exam data model (SRS 3.1.9): an Exam has many ExamSessions; an ExamSession belongs to one UserAccount (the student) and anchors its EventsFlags rows, one RiskScores row, its Recordings rows, and at most one SessionSummary. An Exam references one ProctoringPolicy, and institutions own their users, exams, and retention configuration.

Storage and retention: recordings are written to encrypted object storage buckets, with encryption metadata recorded in the Recordings table. A scheduled job purges expired rows from every table and expired objects from storage according to the configured retention periods (SRS 3.6), and each purge is logged in RetentionLog for compliance review.

Recording export (version 01.01). Exporting a completed session recording requires no schema change: the Recordings row for the session supplies the object_key, and object storage generates a short-lived presigned download URL for the proctor. Availability is determined by the presence of a Recordings row whose object still exists; a recording that was never captured, was already purged by the retention job (SRS 3.6), or fails an integrity check surfaces a recording-unavailable error (Section 5.6). Every export is written to AuditLog.

Database viewer and delete operations (version 01.01). The proctor database viewer reads the existing tables (Section 4.2) through the same role-restricted access as the dashboard, scoped to the proctor’s institution. Deletion applies only to mock/test data and uses hard deletion in dependency order - EventsFlags, RiskScores, Recordings (together with their objects in object storage), and SessionSummaries rows are removed before their anchor ExamSession - because the schema defines no soft-delete flags. Every deletion is confirmed, restricted by role, and logged in AuditLog; RetentionLog continues to record only scheduled retention purges. AuditLog, RetentionLog, and Consents are never deletable through this interface, and no production record of any table may be deleted by a proctor. Identifying mock/test records assumes test data is confined to a dedicated test Institution; a per-record marker, if later required, is an open question for the data design.

4.2 Data Dictionary

The major system entities are listed alphabetically below.

Entity	Type	Description
AuditLog	Table	Append-only record of who viewed or acted on flagged events, recordings, and AI summaries, including severity overrides, recording exports, database deletions, and retention actions (SRS 3.5).
Consents	Table	Student biometric-monitoring consent grants and revocations, maintained for FERPA/GDPR compliance (SRS 3.1.6).
EventsFlags	Table	One row per detected behavioral, ML, or reliability event: session_id, student_id, timestamp, signal_type/error_type, confidence, duration, severity, and review status (SRS 3.1.4).
Exam	Table	Schedulable exam definition: exam_id, title, scheduled_window, duration, policy_template_id (SRS 3.1.9).
ExamSession	Table	A single student attempt: session_id, exam_id, student_id, start_time, end_time, status; the anchor record that events, scores, recordings, and summaries attach to (SRS 3.1.9).
Institution	Table	Tenant account representing an institution; owns users, exams, and retention configuration (SRS 3.1.6).
ProctoringPolicy	Table	Reusable configuration of monitored event types, thresholds, and default severities applied to exams (SRS 3.1.7).
Recordings	Table	Reference to an encrypted audio/video object in object storage (object_key, session_id, format, encryption metadata) - media itself is never stored in the relational database; supports proctor playback and export of completed sessions (SRS 3.1.4).
RetentionLog	Table	Record of every retention purge action (table/bucket, count, timestamp) for compliance review (SRS 3.5, 3.6).
RiskScores	Table	Composite per-session risk: session_id, running score, last_updated (SRS 3.1.4).
RosterAssignment	Table	Assignment of a student or roster to an exam, including accommodations such as extended time (SRS 3.1.7).
SessionSummaries	Table	LLM-generated advisory summary: session_id, summary_text, generated_at, model_version (SRS 3.1.5).
UserAccount	Table	A user of any role (Student, Instructor/Proctor, Administrator, Appeals Reviewer): user_id, role, institution_id, auth_provider (SRS 3.1.9).

4.3 Entity Relationship Diagram

Figure 4-1 renders the relational schema of Section 4.2 as an entity-relationship diagram, grouped by the four table groups described in Section 4.1: Identity & Exam Lifecycle, Monitoring, Media & AI, and Governance. Cardinality is shown on each relationship (1, *, or 0..1); dashed lines indicate a logging relationship rather than a foreign-key ownership relationship.

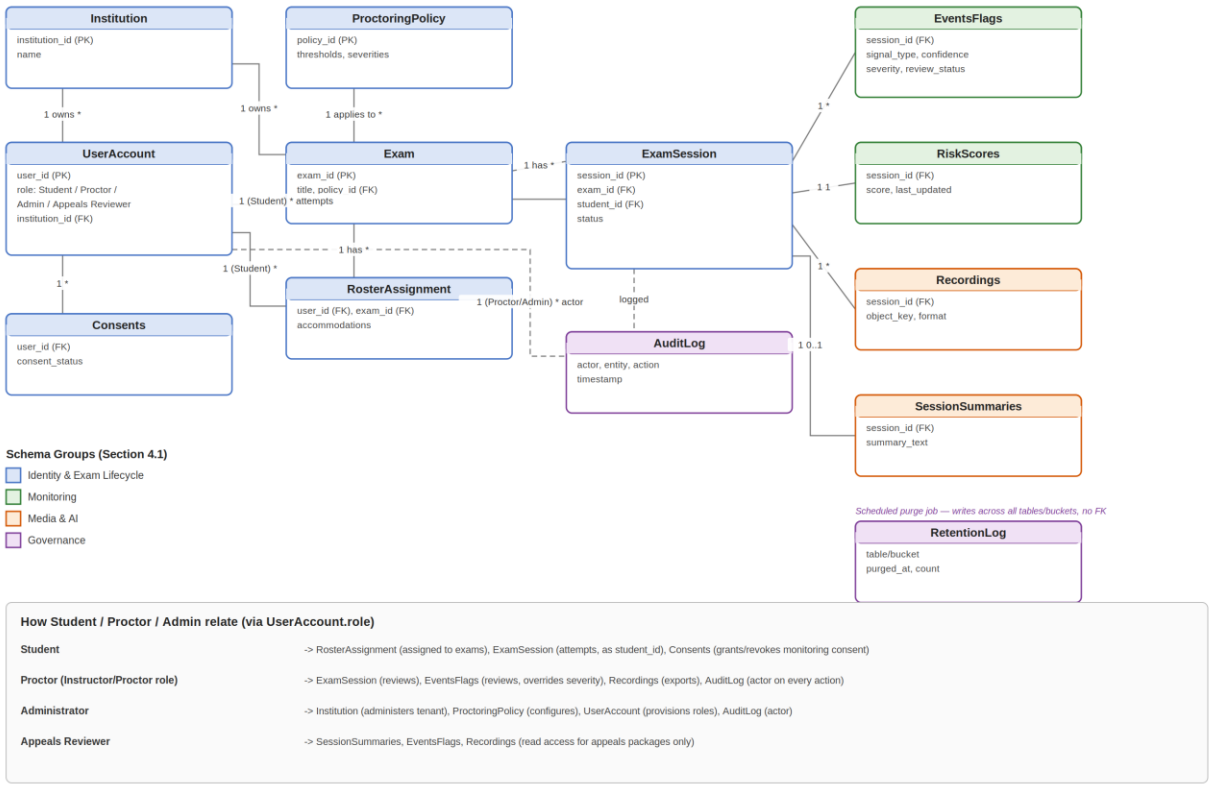


Figure 4-1. Entity-relationship diagram of the schema in Section 4.2, grouped by the four table groups of Section 4.1, with a callout showing how the Student, Proctor, and Administrator values of UserAccount.role relate to the rest of the schema.

As shown in the callout beneath the diagram, UserAccount does not have separate tables per role; role is a single attribute (Section 4.2) that determines which relationships apply to a given account. A Student's UserAccount row is the target of RosterAssignment and ExamSession (as student_id) and owns Consents rows. A Proctor's (Instructor/Proctor role) UserAccount row is the actor on ExamSession review actions, EventsFlags overrides, Recording exports, and the resulting AuditLog rows. An Administrator's UserAccount row administers its Institution, configures ProctoringPolicy, and provisions other UserAccount rows, also logged to AuditLog. This matches the role-gated checks already specified in the PDL of Sections 5.6 and 5.8 (e.g., "if proctor.role not in (Instructor/Proctor, Administrator): deny"), which this diagram makes explicit at the data-model level.

5. COMPONENT DESIGN

This section takes a closer look at what each component does. Procedural descriptions (PDL) are given for the central algorithms; component numbering follows the subsystems of Section 3.2. Components 8 and 9 specify the proctor-side enhancements added in version 01.01; they extend the Proctor Dashboard and introduce no new subsystem.

5.1 Test-Taker Client - Capture, Calibration, and Exam Interface

The client runs in the student's browser during the exam. Before the exam it collects monitoring consent, performs a device check, calibrates the camera and microphone, and captures the head-pose and gaze baseline. During the exam it publishes signals and browser behavior events to the ingestion API while presenting the exam content, timer, and non-punitive integrity notices (SRS 3.1.1, 3.1.8).

CAPTURE_SESSION (session):
open webcam, microphone, and browser behavior hooks
while exam is in progress:
frame <- next video frame; audio <- next audio chunk publish SIGNAL (frame, audio) to the ingestion API if a browser behavior event is detected (tab switch, copy/paste, focus loss):
publish BEHAVIOR_EVENT (event) to the ingestion API if capture is interrupted: publish CONNECTIVITY event if an integrity notice is received: display it (non-punitive), allow self-correction on exit: stop capture, publish SESSION_END

5.2 CV/ML Inference Services - Detection and Classification

Stateless services that estimate head pose and gaze and classify audio and video (SRS 3.1.1, 3.1.2). Thresholds are configurable per proctoring policy; all outputs are advisory flags with confidence values, never final findings.

CALIBRATE_BASELINE (n frames):
collect (pitch, yaw, roll) and gaze_vector estimates for n frames
baseline <- mean of the collected estimates store baseline for the session
MONITOR_HEAD_POSE_GAZE (each frame):

```
    estimate <- (pitch, yaw, roll, gaze_vector, confidence) for the frame
  deviation <- angular distance between estimate and baseline     if
  deviation > threshold and sustained duration > min_duration:
        emit GAZE_DEVIATION or HEAD_POSE_DEVIATION event (confidence, timestamp)
  else: reset the sustained-deviation counter
  CLASSIFY_AUDIO (chunk):
    return class (ambient, typing, whisper, second voice) with confidence CLASSIFY_VISUAL
  (frame):
    return detections (unauthorized object, second person) with confidence
```

5.3 Stream Processor - Fusion and Risk Scoring

The stream processor consumes the queue, fuses multimodal signals, recomputes the composite risk score for the session, persists events and scores, and pushes live updates to proctor dashboards (SRS 3.1.2, 3.1.4).

```
  PROCESS_EVENT (event):
    fused <- FUSE(event) with recent correlated events from other modalities
  severity <- severity_map[event.signal_type] from the exam proctoring policy
  INSERT INTO EventsFlags (session_id, student_id, timestamp, signal_type,
  confidence, duration, severity)
    UPDATE RiskScores SET score = score + weight(severity), last_updated = now
  publish live update (session_id, running score, event) to dashboard subscribers
```

5.4 Storage Layer - Persistence and Retention

Relational tables hold all structured data (Section 4); encrypted object storage holds raw recordings. Media is written directly to object storage, and only a reference pointer is stored in the Recordings table (SRS 3.1.4).

```
  STORE_RECORDING (session, chunk):
    object_key <- upload_encrypted(chunk, session bucket)
    INSERT INTO Recordings (session_id, object_key, format, encryption metadata)
```

5.5 LLM Summarization Service - Advisory Session Narratives

Triggered automatically at exam completion or on demand by a proctor (SRS 3.1.5). It reads only structured EventsFlags rows, builds the prompt from the non-biometric fields, and requests a chronological, timestamped narrative. Every statement in the result must reference a stored event row; the narrative is stored in SessionSummaries and shown as advisory-only. Failures never block proctor review - the dashboard continues with raw flags and the summary is retried later (SRS 3.3).

GENERATE_SUMMARY (session_id):
rows <- SELECT timestamp, signal_type, confidence, duration
FROM EventsFlags WHERE session_id = ? ORDER BY timestamp prompt
<- chronological mapping of rows (time, type, confidence, duration)
- raw media and biometric features are never included narrative <-
call LLM (prompt, require timestamped chronological output) validate:
every statement in narrative references a row in rows
INSERT INTO SessionSummaries (session_id, summary_text, generated_at, model_version)
on failure: log, mark for retry - the dashboard continues with raw flags

5.6 Proctor Dashboard - Review Workflow

Displays live risk scores and flagged events across sessions, plus the AI summary panel labeled as advisory and visually distinct from raw flags (SRS 3.1.3, 3.1.5, 3.2). Every review action writes to the audit trail, and severity overrides retain the original system-assigned value (SRS 3.5, 3.6). It also exports recordings of completed sessions through short-lived presigned URLs; each export is audit-logged (SRS 3.5).

REVIEW_EVENT (proctor, event, action):
match action:
ACKNOWLEDGE_DISMISS: mark event reviewed and non-actionable
ESCALATE: create academic integrity case (event, recording reference, risk context)
ADD_NOTE: append reviewer note to the event
OVERRIDE_SEVERITY: store new severity, retain original system severity
BULK_DISMISS: apply ACKNOWLEDGE_DISMISS to selected low-severity events
INSERT INTO AuditLog (actor, event_id, action, timestamp, before, after) EXPORT_RECORDING
(proctor, session_id):
if proctor.role not in (Instructor/Proctor, Administrator): deny; log attempt
session <- SELECT status, exam_id FROM ExamSession WHERE session_id = ?
if session is missing or session.status != Completed: return recording-unavailable
if session.exam.institution != proctor.institution: deny; log attempt recording <-
SELECT object_key FROM Recordings WHERE session_id = ?
if recording is missing or object fails integrity check: return recording-unavailable
url <- generate_presigned_url(recording.object_key, expiry = short window)
INSERT INTO AuditLog (actor, session_id, action = EXPORT_RECORDING, timestamp)
return url
Failure modes: a recording that was never captured, was purged by retention, or is corrupted returns recordingunavailable without blocking the rest of the dashboard. Large recordings may take time to package and download; the dashboard shows export progress, and the download uses the short-lived presigned URL so media bypasses the relational database.

5.7 Exam Lifecycle Management and Retention Job

Manages the exam state machine - Draft, Scheduled, In Progress, Completed, Archived - gating the actions available to instructors and students in each state (SRS 3.1.7). It also runs the scheduled retention job that purges expired data from all tables and object storage per the configured retention periods (SRS 3.1.4, 3.6).

```
TRANSITION (exam, new_state):
   if new_state not in lifecycle_map[exam.state]: reject the transition
exam.state <- new_state;  publish state-change event
PURGE_RETENTION (now):          // scheduled job     for each
table and object-storage bucket:
      delete rows/objects older than retention_period[type]
INSERT INTO RetentionLog (type, purged_at, count)
```

5.8 Proctor Database Viewer and Data Management - Mock/Test Data Administration

The Proctor Dashboard includes a read-only database viewer over the existing tables (Section 4.2). A proctor can browse, search, filter, and inspect record details across the institution's records; viewing never modifies a record. A guarded delete operation is available for mock/test data only: production records are never deletable through this interface, and the scheduled retention job remains the only production deletion path (SRS 3.6). Every delete requires role authorization, an explicit confirmation, and an AuditLog entry (SRS 3.5).

```
VIEW_RECORDS (proctor, entity, search, filter):
   if proctor.role not in (Instructor/Proctor, Administrator): deny; log attempt
rows <- SELECT * FROM entity WHERE institution = proctor.institution
return rows matching search and filter criteria          // read-only
DELETE_RECORD (proctor, entity, record_id):
   if proctor.role not in (Instructor/Proctor, Administrator): deny; log attempt
if record does not belong to a test Institution: deny; log attempt     require
explicit confirmation from the proctor     if entity = ExamSession: delete
dependents in dependency order:
      EventsFlags, RiskScores, Recordings (and their objects in object storage),
      SessionSummaries
   DELETE FROM entity WHERE id = record_id
   INSERT INTO AuditLog (actor, entity, record_id, action = DELETE, timestamp)
on referential-integrity failure: roll back the transaction, return error, log failure
```

Deletable and protected records. A proctor may delete mock/test records of UserAccount (test student and proctor accounts), Exam (mock exams), and ExamSession together with its dependent EventsFlags, RiskScores, Recordings, and SessionSummaries rows. AuditLog, RetentionLog, and Consents are never deletable through this interface; Institution, ProctoringPolicy, and RosterAssignment are viewable but not deletable. Soft deletion

is not used: the schema has no soft-delete columns, and mock/test data has no retention value; hard deletion in dependency order preserves referential integrity, with the AuditLog entry remaining as the permanent record of the deletion. Identification of mock/test records assumes test data is confined to a dedicated test Institution; a per-record marker, if later required, is an open question for the data design (Section 4.1).

5.9 Proctor Dashboard - Optional Live View (Future Enhancement)

Optional feature. This component is a future enhancement and is not part of the core build. It would let a proctor identify a student whose ExamSession status is Active from the session list, open a live view from the session detail, and watch the student's camera while the test is in progress. Because continuously streaming test sessions may cause lag, high bandwidth usage, and server load, the mechanism is intentionally left open; the design documents candidate approaches without committing to one (Section 3.3): WebRTC peer-to-peer streaming between the test-taker client and the proctor dashboard; WebSocket-based frame push from the capture client through the existing ingestion path; or periodic screenshots/snapshots (for example, one frame every few seconds) as a lightweight fallback. A final decision is deferred until per-session bandwidth and server load are measured.

```
OPEN_LIVE_VIEW (proctor, session_id):          // optional; not part of the core build
if proctor.role not in (Instructor/Proctor, Administrator): deny; log attempt     session
<- SELECT status FROM ExamSession WHERE session_id = ?     if session.status !=
Active: reject with session-not-active     stream <- candidate mechanism: WebRTC
peer feed | WebSocket frame push
                                | periodic snapshots (one frame per N seconds)     display stream in
the session detail view     on network loss or degraded bandwidth: stop the stream, fall
back to the risk-score view     log view session in AuditLog
```

Performance, network, and scalability considerations. A live view adds outbound traffic for every viewed session; N concurrent proctor views multiply egress, and continuous streaming introduces latency and server load. Periodic snapshots bound this cost at one frame per interval per view and are the least expensive option. The capture client already publishes structured signals and stores recordings, so a live view would reuse those paths or add a parallel low-rate channel, and it must degrade gracefully (for example, to the risk-score view) when the network is unstable. Security and privacy: live views are proctor-only, scoped to the proctor's institution, and audit-logged like other proctor actions; the student's monitoring consent already covers captured video (SRS 3.1.6), and a live view displays the same camera stream that is already being recorded. Limitations: live monitoring does not replace flag review; it is an aid, remains advisory, and is explicitly out of scope for the core build.

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The system has four principal web-based interfaces, one per role: the Student Exam Interface, the Proctor Dashboard, the Instructor/Administrator Console, and the Reports and Appeals view.

Student flow. A student signs in (institutional SSO or username/password), reviews and grants monitoring consent, and completes check-in: a device check, camera and microphone calibration, and baseline capture. The exam interface then presents exam content and a timer while proctoring capture runs unobtrusively in the background. When a recoverable issue is detected, the student sees a non-punitive integrity notice (for example, "face not visible") and is given the chance to self-correct. A technical-support button opens contact with a live proctor without pausing or invalidating ongoing monitoring (SRS 3.1.8).

Proctor flow. The dashboard presents prioritized risk scores and flags across all monitored test-takers so a single proctor can monitor many simultaneous sessions (SRS 3.2). Selecting a session shows each flagged event with its context - timestamp, signal type, confidence, and a snippet or thumbnail - so the proctor can judge quickly without replaying the full session. The AI summary panel appears alongside, clearly labeled as AIgenerated and advisory, never in place of the raw flags. Review actions are acknowledged and dismissed, escalated, annotated, severity-overridden, or bulk-reviewed, and every action is logged. For a completed session the proctor can also play back the recording and export it for offline review or appeals (Completed session - Session detail - Recording - Export Video), and a database viewer (Database Viewer - Records - View / Search / Filter - Delete) lets the proctor inspect records and delete mock/test data only, with confirmation and audit logging. An optional live view of a student actively taking a test (Active session - Live View - View Student) is documented as a future enhancement (Section 5.9) and is not part of the core dashboard.

Instructor/administrator flow. The console provisions institutions and rosters, creates and configures exams (window, duration, allowed materials, proctoring policy template), assigns rosters with accommodations, and advances exams through the lifecycle states. Administrators manage roles, tenants, SSO, consent, and retention configuration (SRS 3.1.6, 3.1.7).

Feedback and status. Every review action is confirmed and written to the audit trail. The interfaces surface system status, including degraded modes such as recording-only operation when a real-time ML service is unavailable (SRS 3.3), and report-generation progress in the Reports view. Export, view, and delete actions are confirmed before they run, surface success or failure messages (for example, recording-unavailable), and write to the audit trail.

6.2 Screen Images

Figures 6-1 and 6-2 are wireframe sketches of the two primary screens: the student exam interface and the proctor dashboard. The proctor-side objects added in version 01.01 - the recording export action, the optional live view, and the database viewer - extend the existing Proctor Dashboard wireframe of Figure 6-2 and are described textually in Section 6.3; no new wireframes are introduced in this version.



Figure 6-1. Student exam interface: exam content and timer with the monitoring status sidebar, integrity notice, and technical-support action.

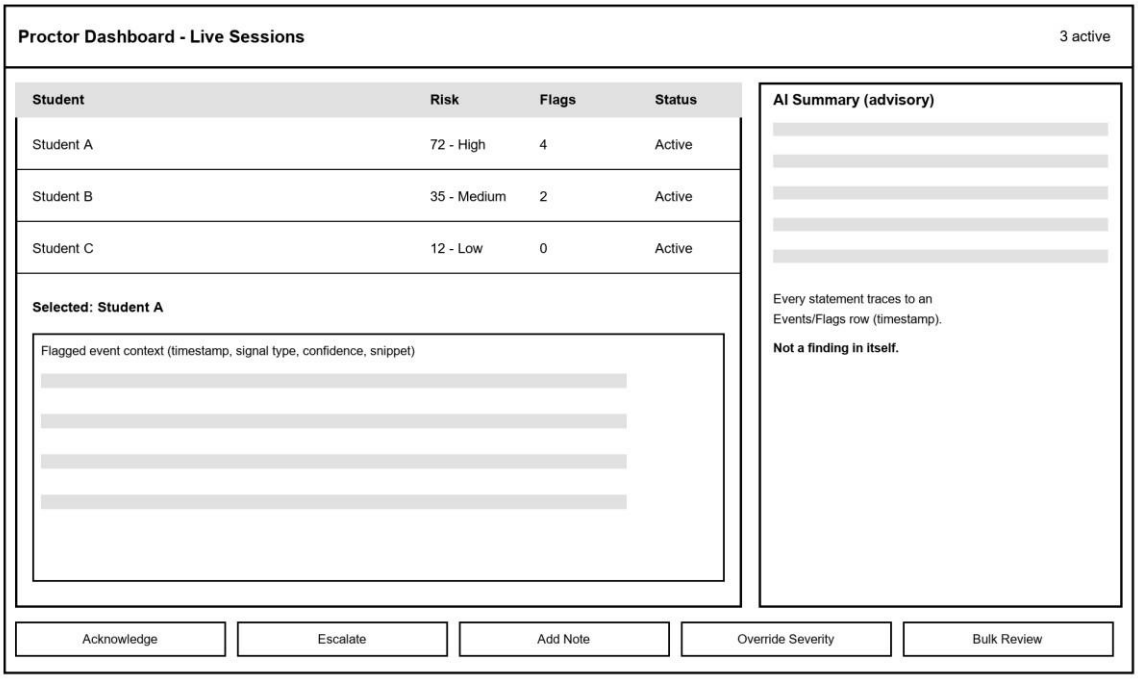


Figure 6-2. Proctor dashboard: live session table with prioritized risk scores, flagged-event context, the advisory AI summary panel, and the review action bar.

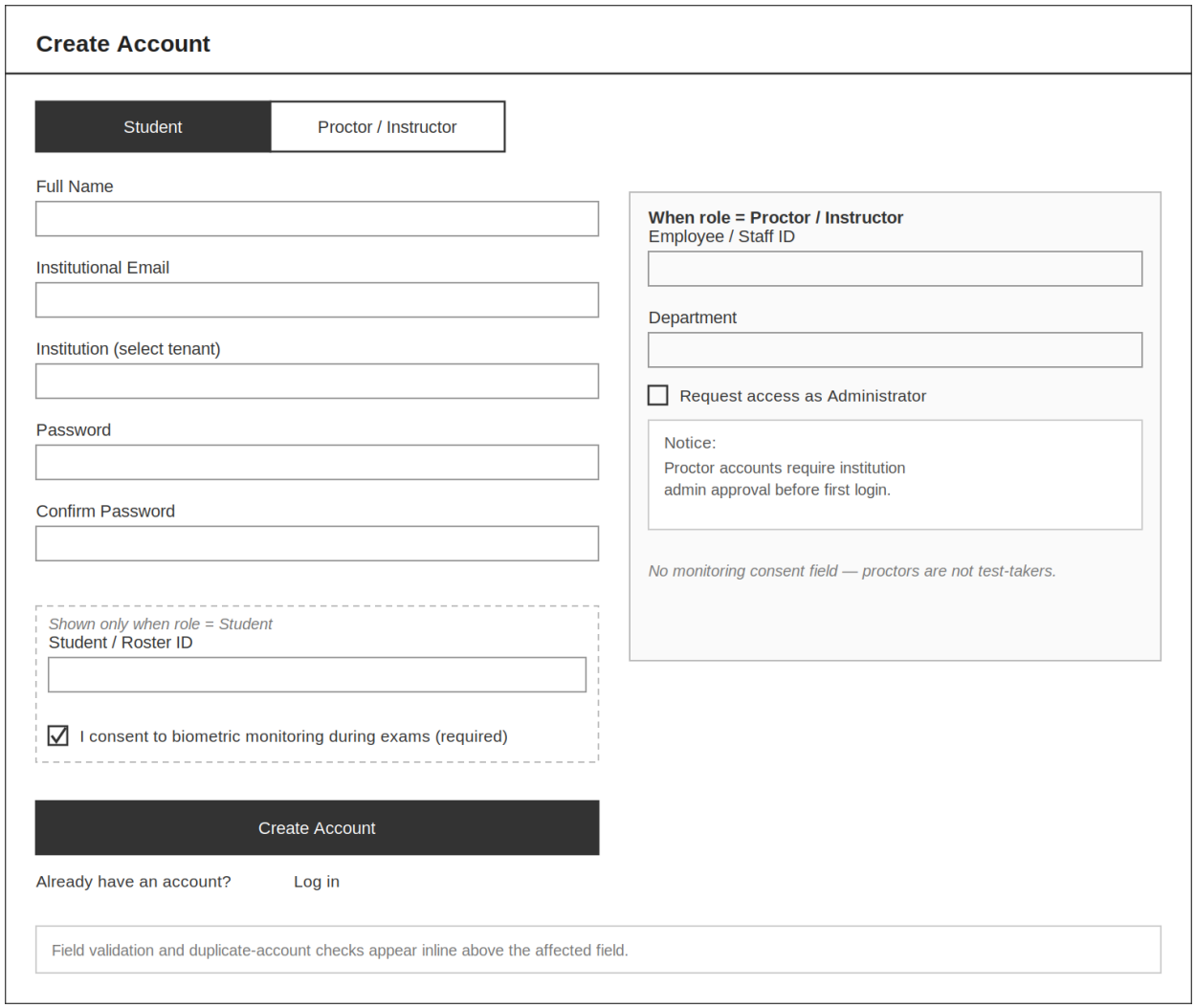


Figure 6-3. Registration page: shared account fields with a Student/Proctor role toggle. The Student role adds a roster ID field and a required monitoring-consent checkbox; the Proctor/Instructor role adds staff identification and a note that the account is subject to institution admin approval before first login (SRS 3.1.6).

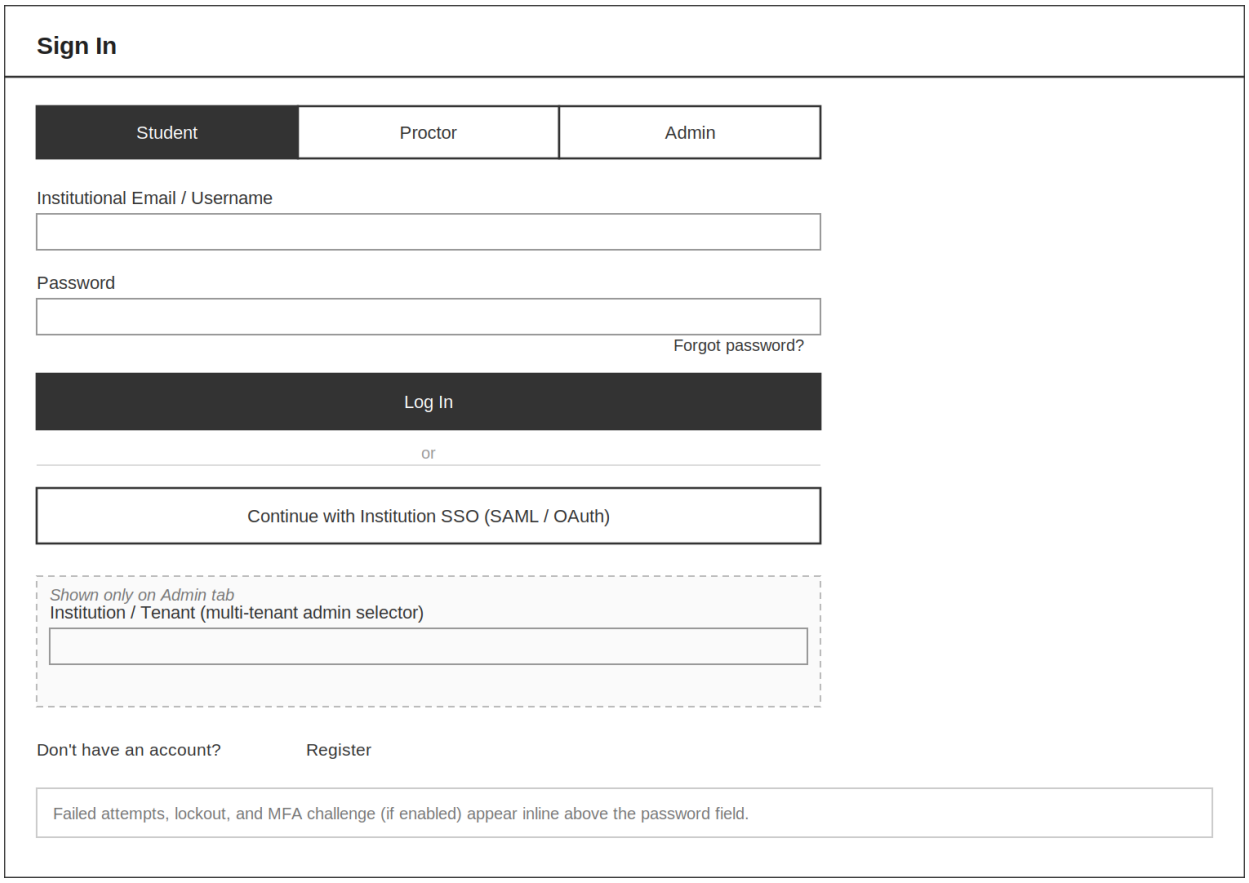


Figure 6-4. Login page: a single sign-in form shared by Student, Proctor, and Admin, selected by tab. Institution SSO (SAML/OAuth) is offered as an alternative to password login for all roles; the Admin tab adds an institution/tenant selector since administrators may be scoped to more than one tenant.

6.3a Screen Objects and Actions

Object	Screen	Actions
Consent and calibration panel	Student check-in	Review and grant or revoke monitoring consent; run camera and microphone check; capture baseline; continue to the exam.
Exam content pane and timer	Student exam interface	Answer questions; view the countdown; submit the exam.
Integrity notice banner	Student exam interface	Read the non-punitive notice; self-correct (for example, return face to camera); dismiss.
Technical support button	Student exam interface	Flag a technical issue to a live proctor/support contact without pausing proctoring.

Session list and risk score tiles	Proctor dashboard	Sort and filter sessions by risk; open a session detail view.
Flagged event card	Proctor dashboard	View event context (timestamp, signal type, confidence, snippet); acknowledge and dismiss; escalate to an academic integrity case; add a reviewer note; override severity; bulk-review low-severity flags.
AI summary panel	Proctor dashboard	Expand or collapse the advisory narrative; crossreference each statement against the underlying Events/Flags rows.
Recording player	Proctor dashboard	Play and seek session recordings; jump to a flagged event timestamp; export/download the recording of a completed session.
Live view panel (optional)	Proctor dashboard	Open the live view of an active session; pause or close the stream; display the student camera feed or periodic snapshots with a degraded-mode indicator (future enhancement, Section 5.9).
Database viewer	Proctor dashboard	Browse, search, filter, and inspect records across the existing tables; open record details.
Delete action (mock/test data)	Proctor dashboard (database viewer)	Delete a permitted mock/test record after explicit confirmation; show success or failure; the deletion is audit-logged.
Exam configuration form	Instructor/administrator console	Create and edit exams; set window, duration, and allowed materials; attach a proctoring policy template; assign rosters and accommodations; change lifecycle state.
Report center	Instructor/administrator console	Generate the session integrity report, class/cohort analytics, appeals package export, and audit/compliance report.

6.3b Screen Objects and Actions (Addendum)

Object	Screen	Actions
Role toggle (Student / Proctor)	Registration page	Switch which role-specific fields are shown; does not change the shared fields already entered.
Registration form	Registration page	Enter full name, institutional email, institution, and password; submit to create an account pending the applicable approval/consent rule for the selected role.

Monitoring consent checkbox	Registration page (Student role)	Grant biometric-monitoring consent; required to submit a Student registration (SRS 3.1.6).
Proctor approval notice	Registration page (Proctor role)	Read-only notice that the account requires institution admin approval before first login.
Role tabs (Student / Proctor / Admin)	Login page	Select which role is signing in; determines whether the institution/tenant selector is shown.
Login form	Login page	Enter institutional email/username and password; submit to authenticate.
Institution SSO button	Login page	Authenticate via the institution's SAML/OAuth identity provider instead of a password.
Institution/tenant selector	Login page (Admin tab)	Choose which institution tenant to administer for this session.

7. REQUIREMENTS MATRIX

The following matrix traces the requirements of the SRS (v1.2) to the components described in Sections 3 and 5 and the data design in Section 4. Functional requirements are listed first, followed by the non-functional requirements. The matrix is followed by the SDD-defined requirements added in version 01.01 for the proctor-side enhancements; these follow the same traceability format but are not SRS (v1.2) requirements.

SRS Requirement	Satisfying Component(s)	SDD Section(s)
3.1.1 Image Processing (Control Accuracy)	CV/ML Inference Services; Test-Taker Client	3.1, 3.2, 5.2
3.1.2 Machine Learning (Data Accuracy)	CV/ML Inference Services; Stream Processor	3.1, 3.2, 5.2, 5.3
3.1.3 Behavioral and Session Monitoring	Test-Taker Client; Proctor Dashboard; Storage Layer	3.2, 5.1, 5.6, 6.2
3.1.4 Data Pipeline and Storage	Ingestion API and Message Queue; Stream Processor; Storage Layer	3.1, 4.1, 5.3, 5.4
3.1.5 AI Summary Generation	LLM Summarization Service	3.1, 3.2, 5.5

3.1.6 User and Account Management	User and Account Management	3.2, 6.1
3.1.7 Examination Management	Examination Management and Online Exam Delivery	3.2, 5.7
3.1.8 Online Examination (Student Experience)	Test-Taker Client; Examination Management and Online Exam Delivery	3.2, 5.1, 6.2
3.1.9 Core Exam Data Model	Storage Layer	4.1, 4.2
3.1.10 Examination and Proctoring Reports	Reporting and Compliance Service	3.2, 6.1
3.2 Usability	Proctor Dashboard; Test-Taker Client	5.1, 5.6, 6.1
3.3 Reliability and Availability	Ingestion API and Message Queue; Stream Processor; LLM Summarization Service	3.3, 5.3, 5.5
3.4 Performance	Ingestion API and Message Queue; Stream Processor; CV/ML Inference Services	3.1, 3.3, 5.2, 5.3
3.5 Security	Storage Layer; User and Account Management; AuditLog	3.3, 4.1, 5.4, 5.6
3.6 Design Constraints	All components - human-in-the-loop and LLM data minimization enforced in Proctor Dashboard, Stream Processor, and LLM Summarization Service	3.3, 4.1, 5.5
3.7 Interfaces	Test-Taker Client; Proctor Dashboard; Storage Layer; LLM Summarization Service	3.1, 3.2, 4.1

SDD-defined requirements (version 01.00).

SDD Requirement	Satisfying Component(s)	SDD Section(s)
SDD-1.1 Completed Session Recording Export	Proctor Dashboard; Storage Layer	3.1, 3.2, 3.3, 4.1, 5.6, 6.3

SDD-1.2 Optional Live Student View (future enhancement)	Proctor Dashboard (Section 5.9); Test-Taker Client	3.2, 3.3, 5.9, 6.1, 6.3
SDD-1.3 Proctor Database Viewer	Proctor Dashboard; Storage Layer	3.2, 4.1, 4.2, 5.8, 6.3
SDD-1.4 Proctor Delete Operations (mock/test data only)	Proctor Dashboard; Storage Layer; AuditLog	3.2, 3.3, 4.1, 5.8, 6.3
SDD-2.1 Live-View Performance and Network Budgets (optional feature)	Proctor Dashboard (Section 5.9)	3.3, 5.9
SDD-2.2 Security and Auditability of Proctor-Side Enhancements	Proctor Dashboard; Storage Layer; AuditLog	3.3, 4.1, 5.6, 5.8, 5.9